UR Nyarugenge

College of Science and Technology

Computer and Software Engineering

Level 3

Mobile application assignment

1.

① which dependencies did you use and why?

Dependencies used:

- flutter (SDK): the core flutter framework for building the UI and app structure
- http: ^1.2.0: for making HTTP requests to the JSONPlaceholder API (e.g. GET, POST, PUT, DELETE operations in PostService)
- cupertino_icons: ^1.0.8: Provides iOS-style icons (used in the app bar and buttons for a polished look).

Why these dependencies;

- flutter is essential as it's the foundation of the app
- http was chosen for its simplicity and reliability in handling REST API calls, including timeouts and headers. its lightweight and doesn't require additional set up
- cupertino_icons adds native iOS icons to improve the UI's visual consistency, especially for buttons like refresh and add.

The dependencies: section showing http: ^1.2.0 and cupertino_icons: ^1.0.8

```
7
8  dependencies:
9    flutter:
10     sdk: flutter
11   http: ^1.2.0
12       coco, 41 minutes ago • Add assignment codes
13   cupertino_icons: ^1.0.8
14
```

(8) Generally, how can you handle API exception (network errors, invalid data)

General Approach to handling API exceptions:

- Custom exception classes: Define specific exception eg. NetworkException, ServerException, to categorize errors such as network issues, server errors, parsing failures
- Try-catch Blocks: Wrap API calls in try-catch to catch and rethrow or handle exceptions
- Error Propagation: Rethrow custom exceptions to let the UI layer (e.g. FutureBuilder) handle them via snapshot.hasError.
- UI Handling: Use FutureBuilder to check snapshot.hasError and display user-friendly messages or retry options
- Specific cases:
  - Network errors: Catch SocketException or HttpException and throw NetworkException
  - Invalid Data: Catch ~~Formate~~ FormatException during JSON parsing and throw ParseException.
  - Server Errors: Check HTTP status codes (e.g 404 for not found, 5xx for server issues) and throw ServerException.

This ensures robust error handling, preventing crashes and providing clear feedback to users.

In my code:

- Customer exceptions are defined in api_exception.dart
- In post_service.dart, each API method uses try-catch to handle errors and rethrow custom exceptions
- In the screens (post_list_screen.dart and Post_detail_screen.dart) FutureBuilder checks for errors and shows messages or retry buttons

All custom exception class definitions

(ApiException, NetworkException, ServerException, ParseException, NotFoundException)

```
class ApiException implements Exception {
  final String message;
  final int? statusCode;

  const ApiException(this.message, {this.statusCode});

  @override
  String toString() => 'ApiException: $message (status: $statusCode)';
}
coco, 44 minutes ago | 1 author (coco)
class NetworkException extends ApiException {
  const NetworkException(super.message);

  @override
  String toString() => 'NetworkException: $message';
}
coco, 44 minutes ago | 1 author (coco)
class ServerException extends ApiException {
  const ServerException(super.message, {super.statusCode});

  @override
  String toString() => 'ServerException: $message (status: $statusCode)';
}
coco, 44 minutes ago | 1 author (coco)
class ParseException extends ApiException {
  const ParseException(super.message);

  @override
  String toString() => 'ParseException: $message';
}
coco, 44 minutes ago | 1 author (coco)
class NotFoundException extends ApiException {
  const NotFoundException(super.message) : super(statusCode: 404);

  @override
  String toString() => 'NotFoundException: $message';
}
```

The fetchAllPosts() method showing try-catch blocks handling different exception types

```
Future<List<Post>> fetchAllPosts() async {
  try {
    final response = await http
        .get(Uri.parse(_baseUrl), headers: _headers)
        .timeout(_timeout);

    _checkStatus(response);

    final List<dynamic> jsonList = jsonDecode(response.body) as List<dynamic>;
    return jsonList
        .map((e) => Post.fromJson(e as Map<String, dynamic>))
        .toList();
  } on ApiException {
    rethrow;
  } on SocketException {
    throw const NetworkException(
        'No internet connection. Please check your network.');
  } on HttpException {
    throw const NetworkException('Could not reach the server.');
  } on FormatException catch (e) {
    throw ParseException('Invalid data format: ${e.message}');
  } catch (e) {
    throw ApiException('An unexpected error occurred: $e');
  }
}
```

The _checkStatus() method showing how HTTP status codes map to specific exceptions

```
143
144   void _checkStatus(http.Response response) {
145     final code = response.statusCode;
146     if (code >= 200 && code < 300) return;
147
148     if (code == 404) {
149       throw const NotFoundException('The requested resource was not found.');
150     } else if (code >= 400 && code < 500) {
151       throw ServerException(        coco, 56 minutes ago • Add assignment codes
152         'Client error: ${response.reasonPhrase}',
153         statusCode: code,
154       );
155     } else if (code >= 500) {
156       throw ServerException(
157         'Server error: ${response.reasonPhrase}',
158         statusCode: code,
159       );
160     } else {
161       throw ServerException(
162         'Unexpected response: ${response.reasonPhrase}',
163         statusCode: code,
164       );
165     }
166   }
167 }
```

(2) Explain the Future widget you have used. And the part of the UI that depends on the Future.

Future widgets used:

FutureBuilder: A Flutter widget that builds its UI based on the state of a Future. It handles asynchronous operations (like API calls) by rebuilding the UI when the Future completes (success, error, or loading)

How it works:

- Takes a Future parameter (the async operation)
- Uses a builder function with AsyncSnapshot to check states: connection state. waiting (loading), hasError (error), or hasData (success).
- Automatically rebuilds when the Future changes e.g after refresh

Why used: simplifies async UI updates without manual state management. In your app, its used for fetching posts and handling loading/error states seamlessly.

Parts of the UI that depend on the Future:

- Loading state: shows a spinner and loading posts... text while data is fetched
- Error state: Display error messages (e.g. network issues with a retry button.
- Data state: Renders the list of posts or post details once data is available
- Refresh/Retry: Buttons trigger new Futures, causing the UI to reload.

In my code:

- FutureBuilder <List<Post>> in post_list_screen.dart for the post list
- FutureBuilder <Post> in post_detail_screen.dart for post details and app bar buttons.

The loading state

```
return const Center(
  child: Column(
    mainAxisSize: MainAxisSize.min,
    children: [
      CircularProgressIndicator(),
      SizedBox(height: 16),
      Text('Loading posts…', style: TextStyle(color: Colors.grey)),
    ],
```

The error state (ErrorDisplay widget with retry button)

```
if (snapshot.hasError) {
  final error = snapshot.error;
  String message = 'Something went wrong.';
  if (error is NetworkException) {
    message = error.message;
  } else if (error is ApiException) {
    message = error.message;
  }
```

The success state (ListView showing posts with edit/delete options)

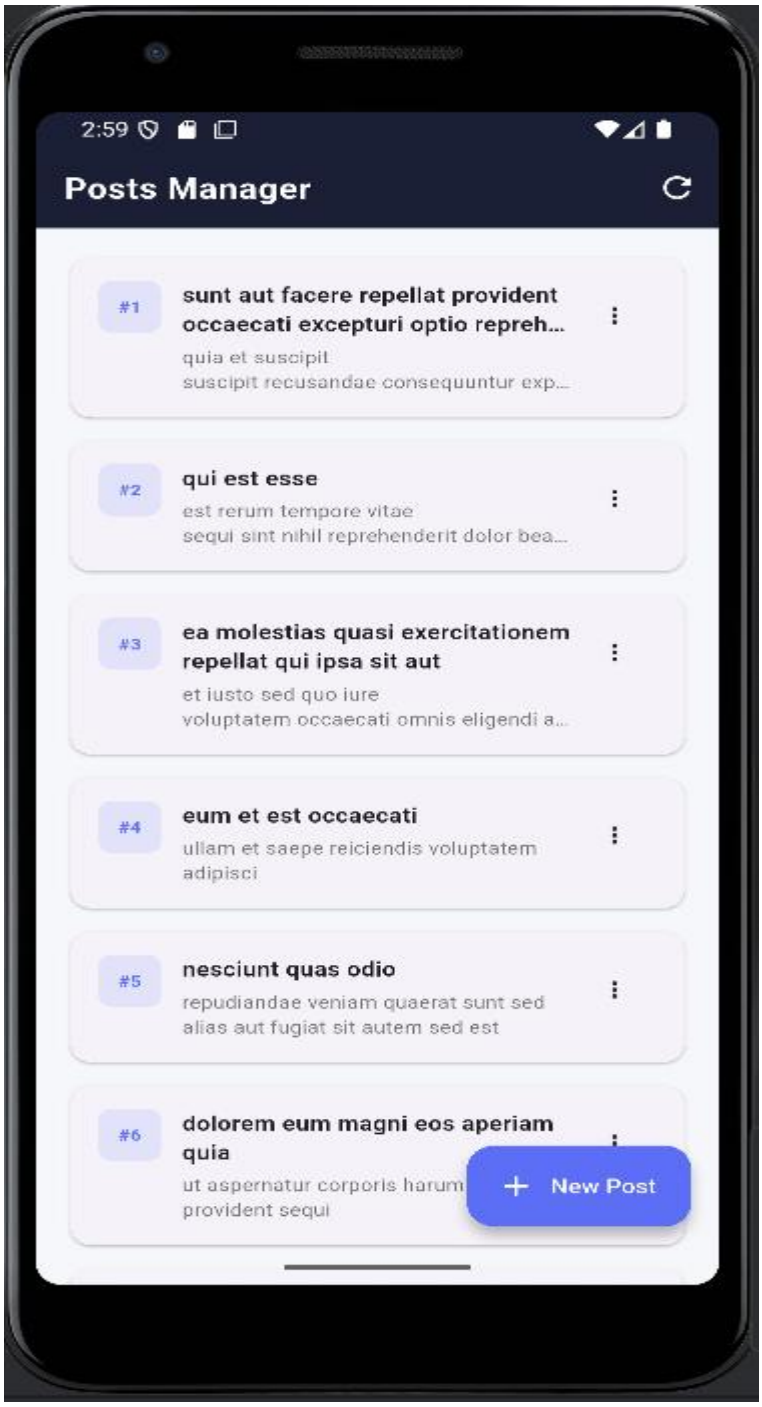
```
132
133
134         return ListView.separated(
135           padding: const EdgeInsets.all(16),
136           itemCount: posts.length,
137           separatorBuilder: (_, __) => const SizedBox(height: 8),
138           itemBuilder: (context, index) {
139             final post = posts[index];
140             return _PostCard(
141               post: post,
142               onTap: () async {
143                 await Navigator.push(
144                   context,
145                   MaterialPageRoute(
146                     builder: (_) => PostDetailScreen(post: post),
147                   ), // MaterialPageRoute
148                 );
149                 _loadPosts();
150               },
151               onDelete: () => _deletePost(post),
152               onEdit: () async {
153                 await Navigator.push(
154                   context,
155                   MaterialPageRoute(
156                     builder: (_) => PostFormScreen(post: post),
157                   ), // MaterialPageRoute
158                 );
159                 _loadPosts();
160               },
161             ); // _PostCard
162           },
```

FutureBuilder controlling Edit/Delete buttons visibility

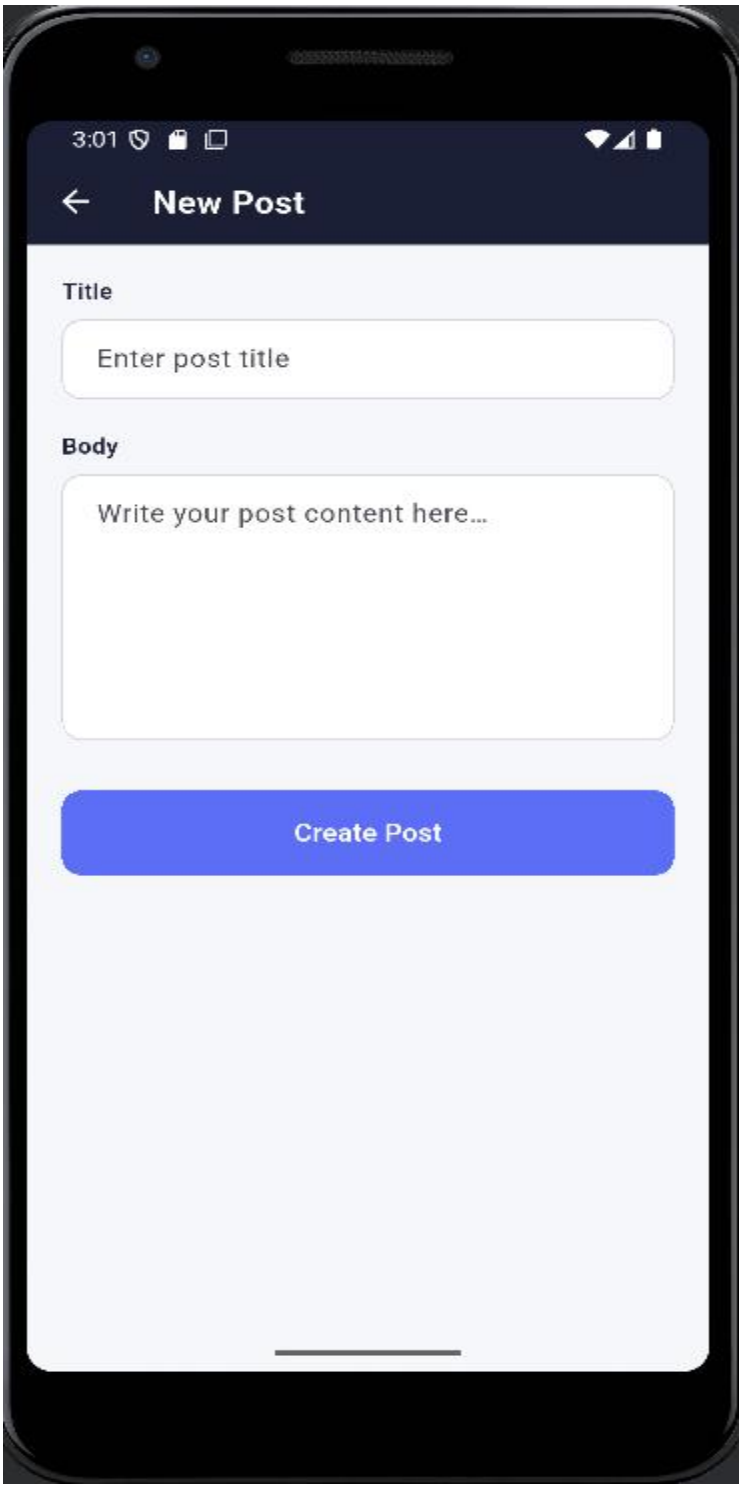
```
@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFF5F7FA),
    appBar: AppBar(
      backgroundColor: const Color(0xFF1A1F36),
      foregroundColor: Colors.white,
      title: const Text('Post Details'),
      actions: [
        // FutureBuilder here controls when the Edit/Delete buttons appear
        FutureBuilder<Post>(
          future: _postFuture,
          builder: (context, snapshot) {
            if (!snapshot.hasData) return const SizedBox.shrink();
            final post = snapshot.data!;
            return Row(
              children: [
                IconButton(
                  icon: const Icon(Icons.edit_outlined),
                  tooltip: 'Edit',
                  onPressed: () async {
                    await Navigator.push(
                      context,
                      MaterialPageRoute(
                          builder: (_) => PostFormScreen(post: post)), // MaterialPageR
                    );
                    setState(() {
                      _postFuture = _service.fetchPost(post.id!);
                    });
                  },
                ), // IconButton
```

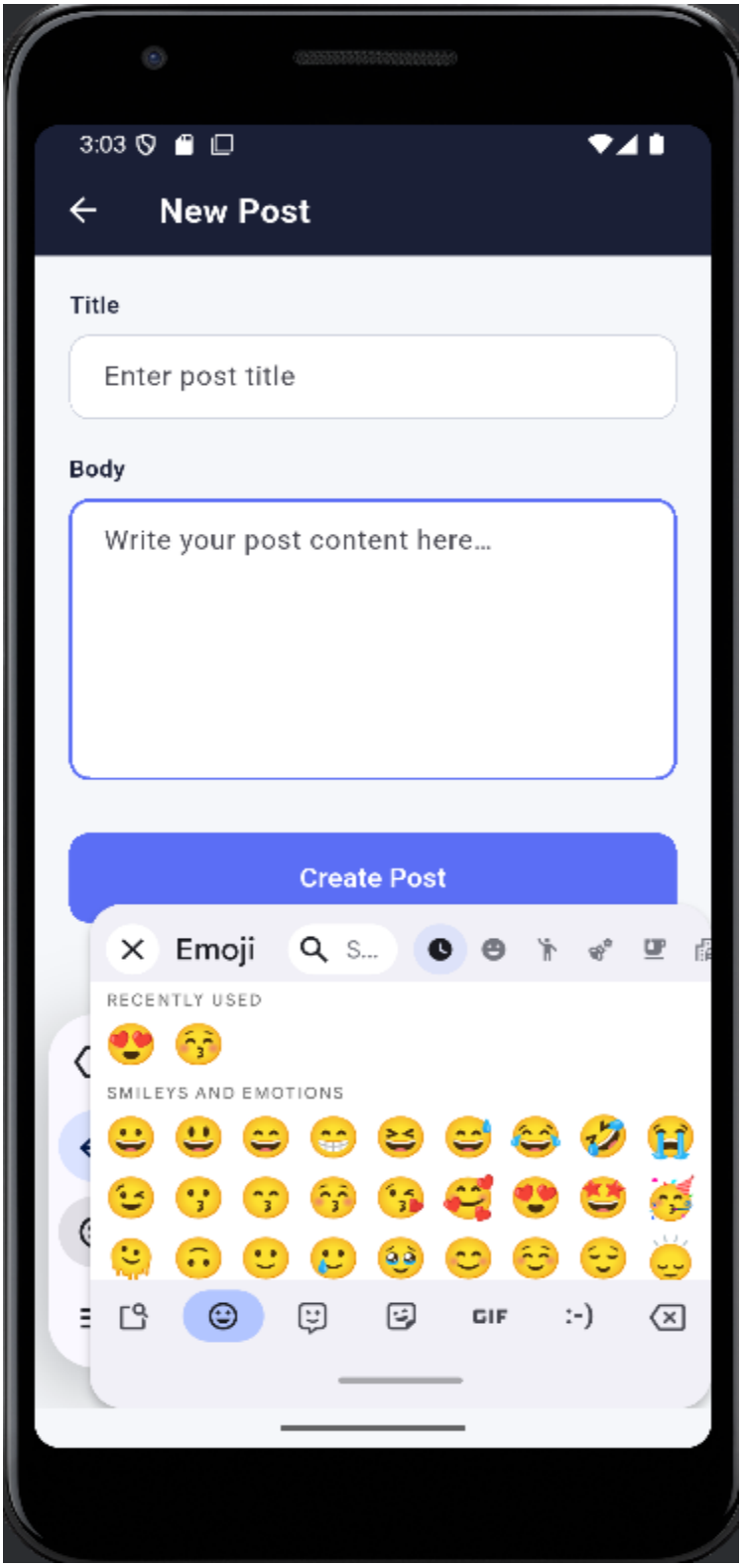
Other screenshots related to how the app is running

Lists of posts

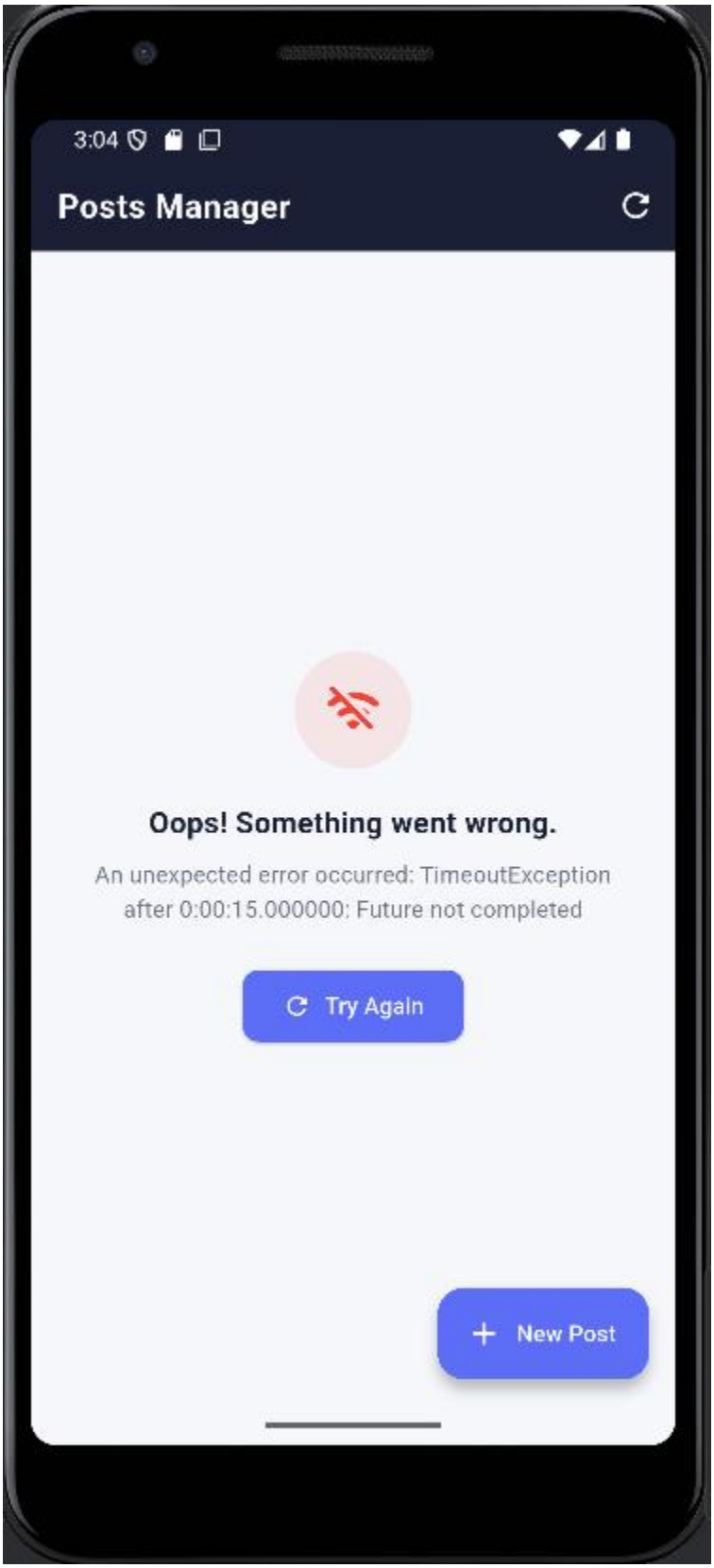


Creating new post

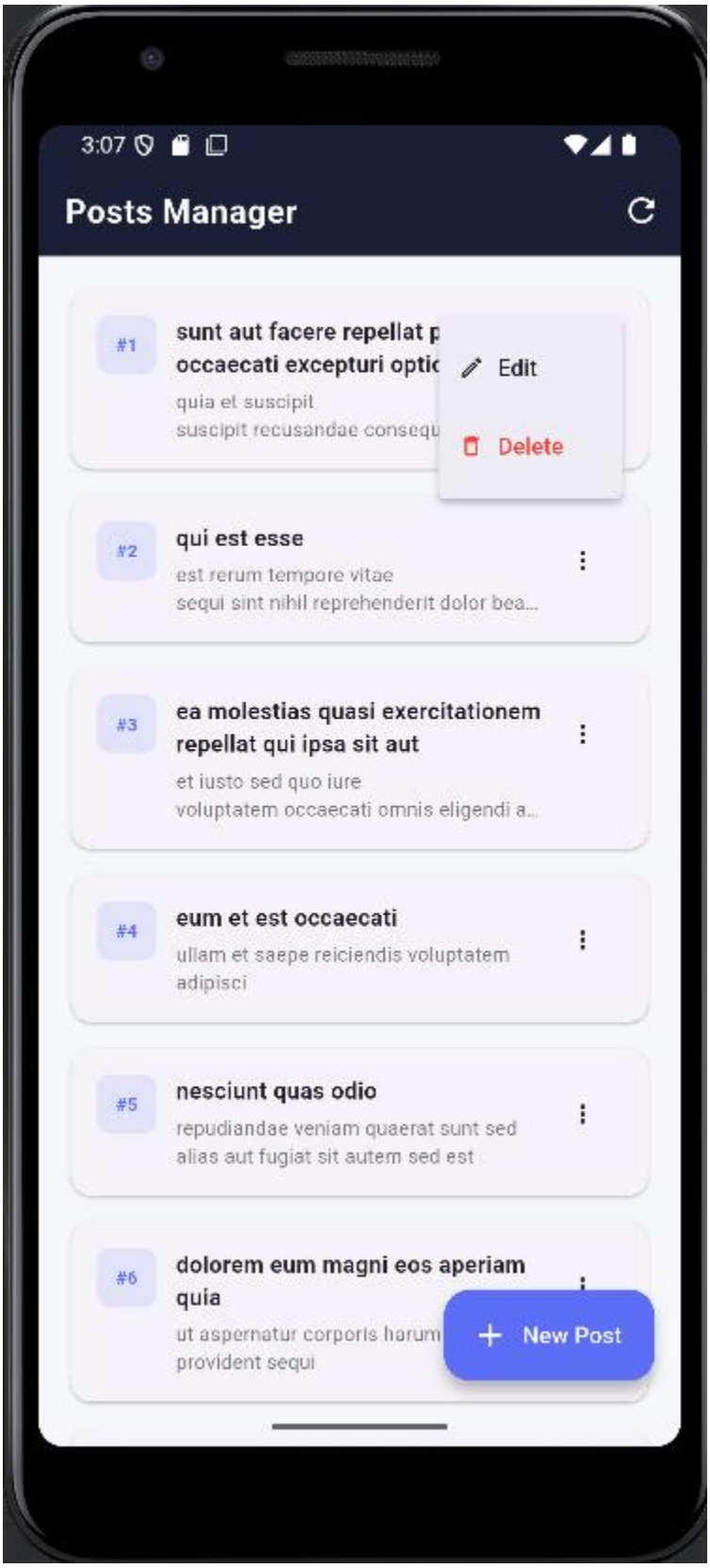



3:03
New Post
Title
Enter post title
Body
Write your post content here...
Create Post
Emoji
S...
RECENTLY USED
SMILEYS AND EMOTIONS
GIF
:-)

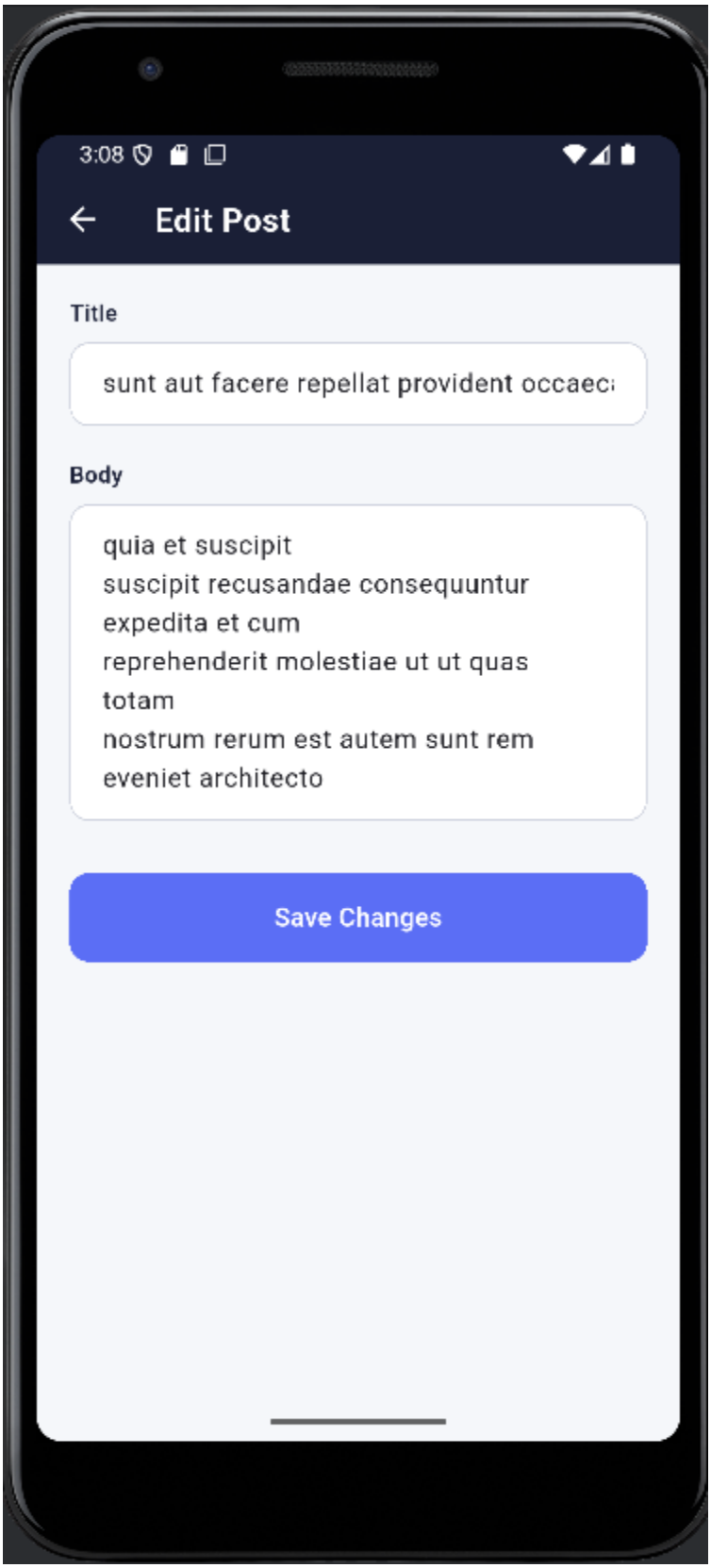
unexpected error happened



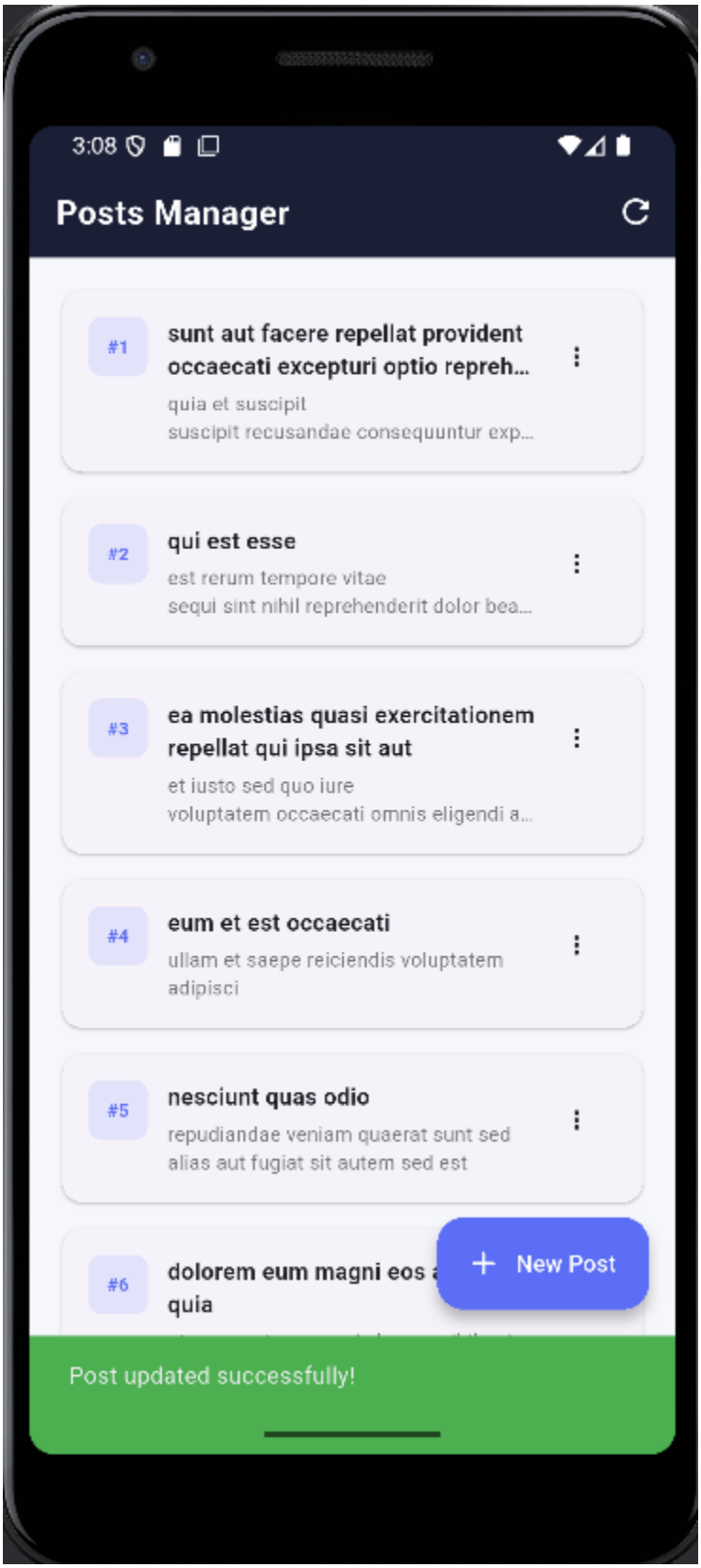
Editing and Deleting



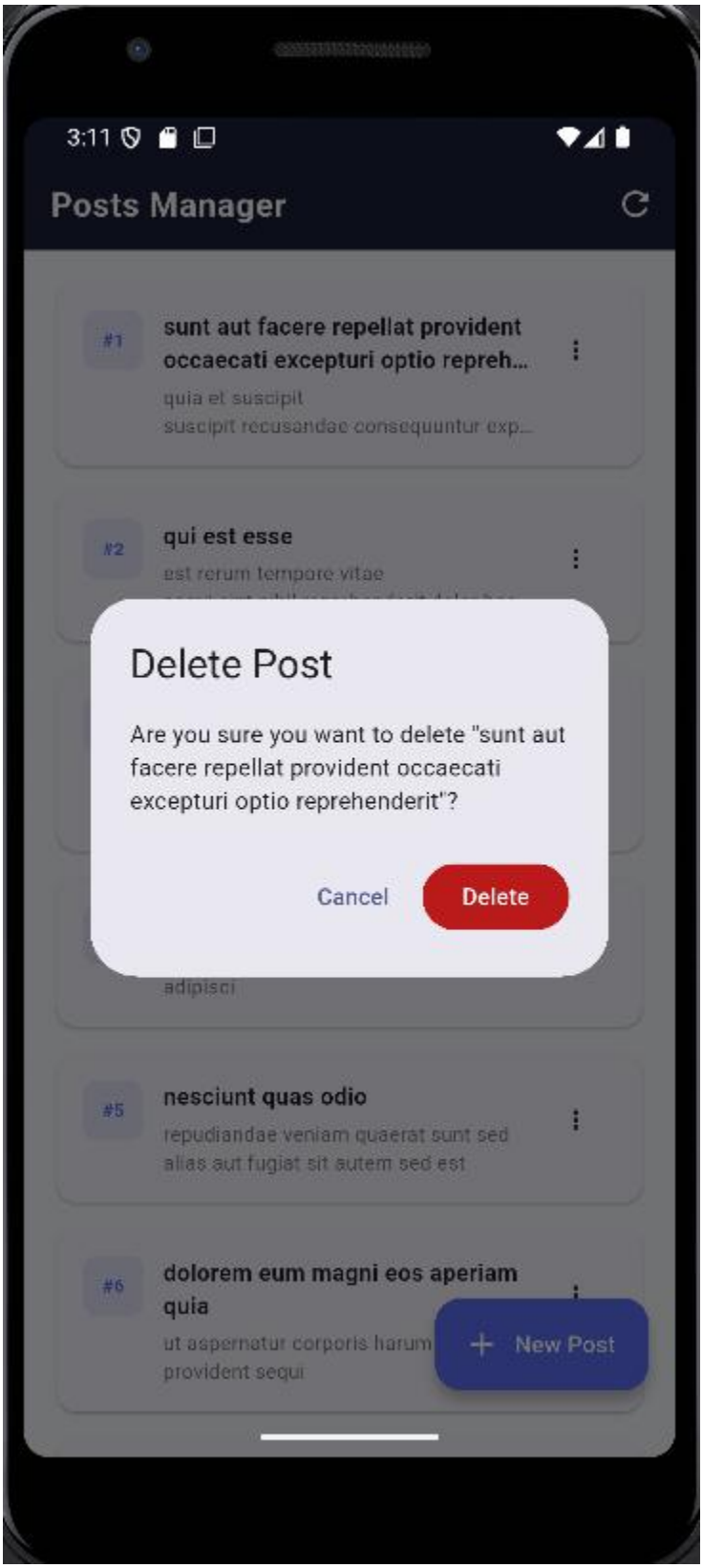
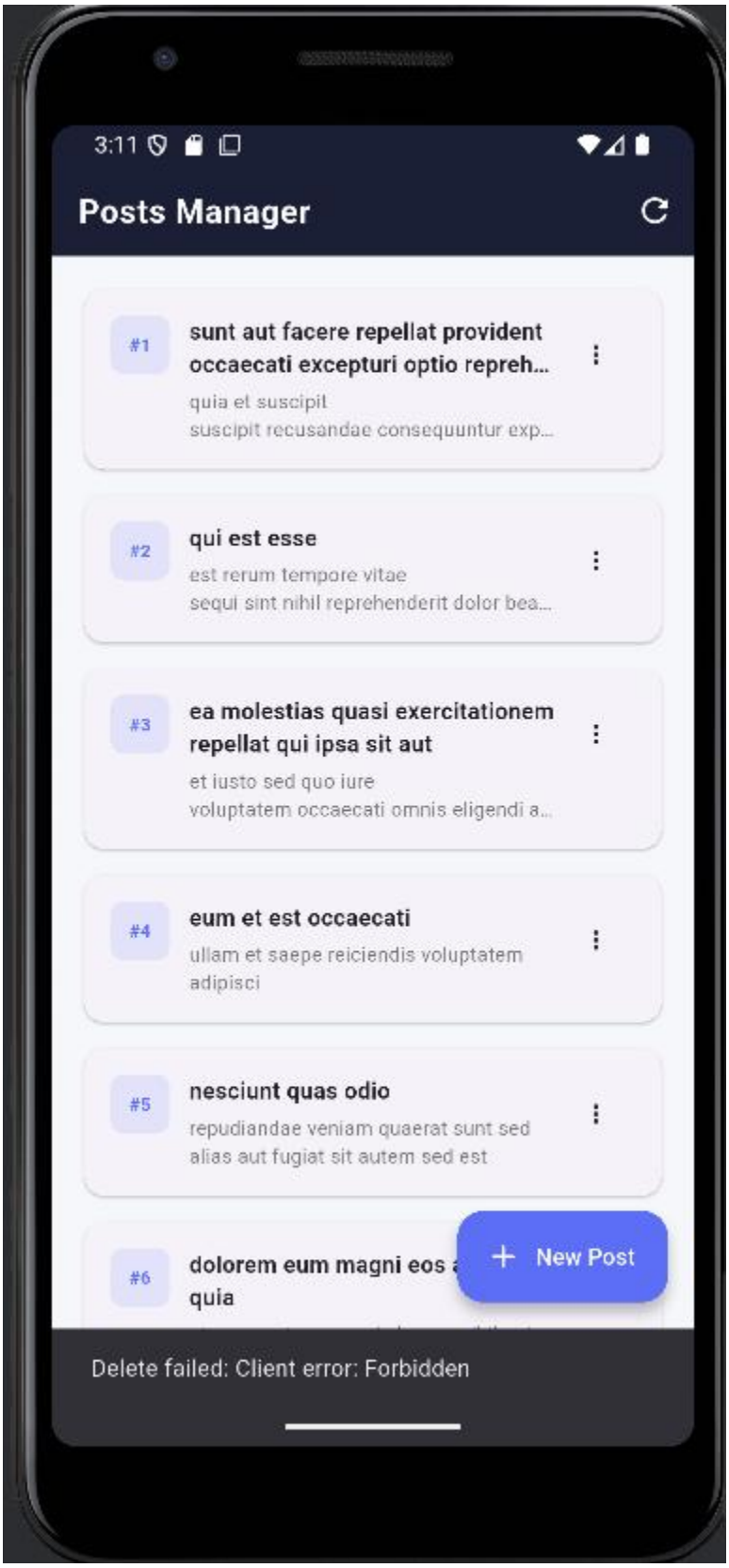
Editing


3:08
Edit Post
Title
sunt aut facere repellat provident occaec
Body
quia et suscipit
suscipit recusandae consequuntur
expedita et cum
reprehenderit molestiae ut ut quas
totam
nostrum rerum est autem sunt rem
eveniet architecto
Save Changes

Successful update



Deleting post



Client error